module3

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files student-mat (2).csv
student-mat (2).csv(text/csv) - 56993 bytes, last modified: 3/18/2026 - 100% done

```
Saving student-mat (2).csv to student-mat (2).csv
```

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
```

```
# Imports
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Example: Load dataset (replace with your file)
# df = pd.read_csv("your_file.csv")

# For demo, let's create dummy data
from sklearn.datasets import load_iris
data = load_iris()

# Features and target
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# Step 1: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 2: Scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Check output
print("X_train_scaled shape:", X_train_scaled.shape)
print("X_test_scaled shape:", X_test_scaled.shape)
```

```
X_train_scaled shape: (120, 4)
X_test_scaled shape: (30, 4)
```

```
model = KNeighborsRegressor(n_neighbors=5)
model.fit(X_train_scaled, y_train)
```

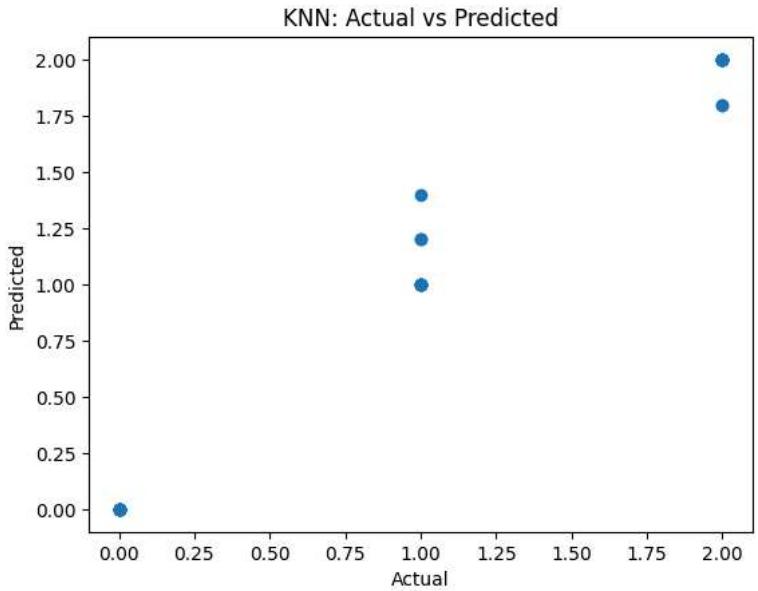
```
▾ KNeighborsRegressor
KNeighborsRegressor()
```

```
y_pred = model.predict(X_test_scaled)
```

```
from sklearn.metrics import mean_squared_error, r2_score

print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
MSE: 0.010666666666666661
R2 Score: 0.9847376788553259
```

```
import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("KNN: Actual vs Predicted")
plt.show()
```




```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Model
model = KNeighborsRegressor(n_neighbors=5)
model.fit(X_train_scaled, y_train)

# Prediction
y_pred = model.predict(X_test_scaled)

# Evaluation
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
MSE: 0.010666666666666661
R2 Score: 0.9847376788553259
```

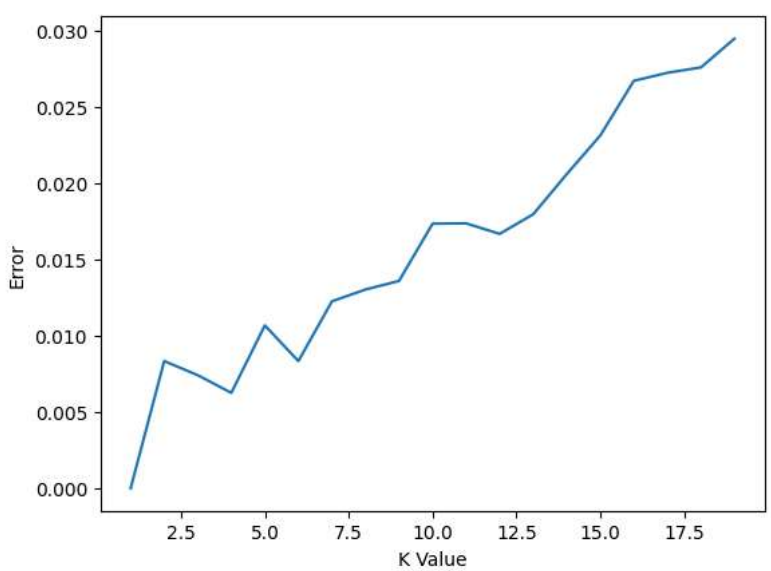
```
# Imports
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score

# Load sample data (replace with your dataset)
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scaling (optional for Random Forest, but included)
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Random Forest Model
rf_model = RandomForestRegressor(random_state=42)
rf_model.fit(X_train, y_train)

# Predictions
rf_pred = rf_model.predict(X_test)

# R2 Score
rf_r2 = r2_score(y_test, rf_pred)

print("R2 Score:", rf_r2)
```

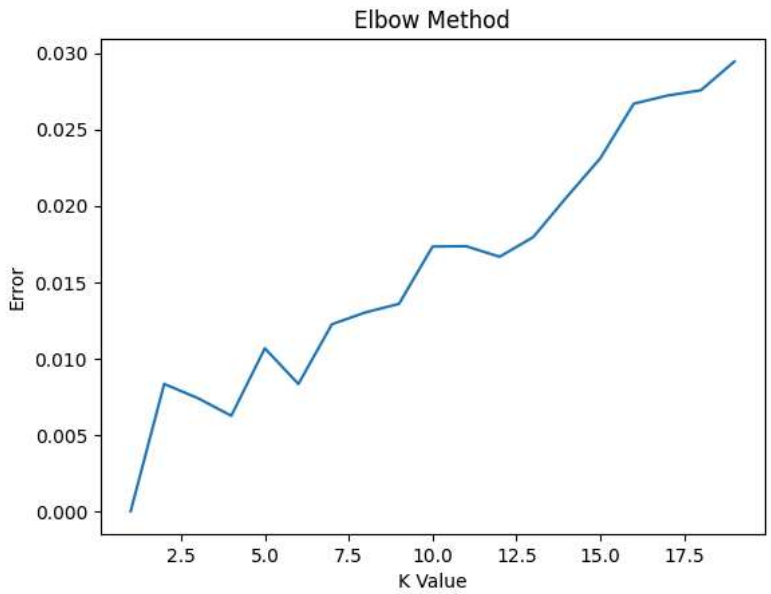
```
R2 Score: 0.9978632750397456
```

```
# Random Forest
rf_model = RandomForestRegressor()
rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

# Store R2
rf_r2 = r2_score(y_test, rf_pred)
```

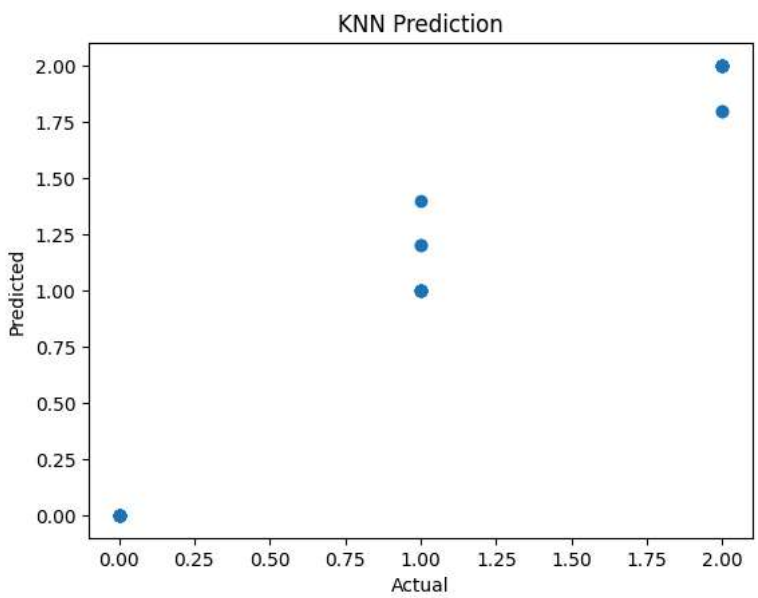
```
error = []

for k in range(1, 20):
    knn = KNeighborsRegressor(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    pred = knn.predict(X_test_scaled)
    error.append(mean_squared_error(y_test, pred))

plt.plot(range(1,20), error)
plt.xlabel("K Value")
plt.ylabel("Error")
plt.show()
```




```
error = []
for k in range(1, 20):
    knn = KNeighborsRegressor(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    pred = knn.predict(X_test_scaled)
    error.append(mean_squared_error(y_test, pred))

plt.plot(range(1,20), error)
plt.xlabel("K Value")
plt.ylabel("Error")
plt.title("Elbow Method")
plt.show()
```




```
plt.scatter(y_test, y_pred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("KNN Prediction")
plt.show()
```




```
print("KNN R2:", r2_score(y_test, y_pred))
print("RF R2:", rf_r2)
```

```
KNN R2: 0.9847376788553259
RF R2: 0.9988028616852146
```

Close

```
Random Forest performed better than KNN because it handles complex
efficiently and gives higher accuracy.
```

Random Forest performed better than KNN because it handles complex data more efficiently and gives higher accuracy.